



Widgets

[View](#)

[View Events Listeners](#)

[TextView](#)

[EditText](#)

[Listeners comunes](#)

[Button](#)

[Toast](#)

[ImageView](#)

[RadioGroup y RadioButton](#)

[Switch](#)

[CheckBox](#)

View

Esta clase representa el componente básico de los componentes de la interfaz de usuario. Una Vista ocupa un área rectangular en la pantalla y es responsable del dibujo y el manejo de eventos. View es la clase base para los widgets , que se utilizan para crear componentes de interfaz de usuario interactivos (botones, campos de texto, etc.). .

Existen algunos tipos de operaciones comunes como:

- **Establecer propiedades:** Las propiedades disponibles y los métodos que las configuran variarán entre las diferentes subclases de vistas.
- **Establecer enfoque:** el marco manejará el enfoque móvil en respuesta a la entrada del usuario. Para forzar el enfoque en una vista específica, llame a [requestFocus\(\)](#).
- **Configurar oyentes:** las vistas permiten a los clientes configurar oyentes que serán notificados cuando suceda algo interesante en la vista. Algunas subclases de views ofrecen oyentes más especializados.
- **Establecer visibilidad:** puede ocultar o mostrar vistas

View Events Listeners

- **setOnClickListener-** Devolución de llamada cuando se hace clic en la vista
- **setOnTouchListener-** Devolución de llamada cuando se arrastra la vista
- **setOnFocusChangeListener-** Devolución de llamada cuando la vista cambia de enfoque
- **setOnGenericMotionListener-** Devolución de llamada para gestos arbitrarios.

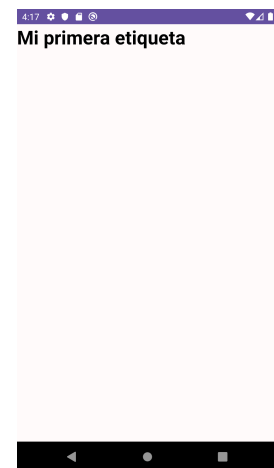


- **setOnHoverListener**- Devolución de llamada al pasar el cursor sobre la vista.
- **setOnKeyListener**- Devolución de llamada para presionar una tecla de hardware cuando la vista tiene foco
- **setOnLongClickListener**- Devolución de llamada para mantener presionada una vista
- **setOnTouchListener**- Devolución de llamada para tocar hacia abajo o hacia arriba en una vista

TextView

El widget más simple es la etiqueta, representada por la clase `TextView`. Se utiliza para formar un texto que normalmente sirve para identificar visualmente otro componente que se sitúa justo al lado. Hay muchas propiedades de la clase `TextView` para formatear el texto de la etiqueta. Por ejemplo:

- **android:typeface** permite cambiar el tipo de fuente (normal, sans, serif, monospace) para la etiqueta.
- **android:textStyle** permite seleccionar el estilo de la letra (cursiva, negrita, o cursiva y negrita).
- **android:textColor** permite seleccionar el color de la etiqueta en formato RGB hexadecimal.
- **android:textSize** permite especificar el tamaño de la fuente, por ejemplo 20dp (20 density-independent pixels)



```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/lytContenedorSaludo"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <TextView android:id="@+id/main_etiqueta1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/etiqueta1"
        android:typeface="normal"
        android:textStyle="bold"
        android:textColor="@color/black"
        android:textSize="30dp"
    />
</LinearLayout>
```

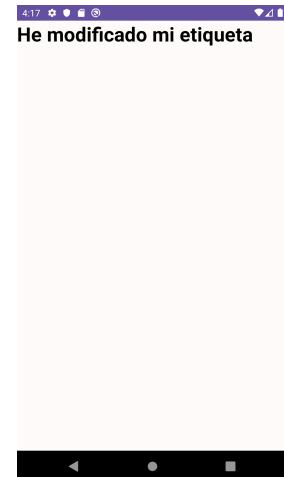


```
class MainActivity : AppCompatActivity() {

    private lateinit var primEtiq: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        primEtiq =
            findViewById(R.id.main_etiqueta1)
        primEtiq.setText("He modificado mi
            etiqueta")
    }
}
```



EditText

La clase EditText representa el widget más popular junto con las etiquetas y los botones. Es una subclase de la clase TextView (por lo que hereda todas sus propiedades, algunas vistas en el apartado anterior) y tiene otras muy interesantes propiedades como por ejemplo:

- android:inputType, que permite seleccionar entre varios tipos de teclado que se mostrarán cuando se esté introduciendo texto:
 - "text", para textos normales,
 - "textEmailAddress" para incluir el carácter @,
 - "textPassword" para contraseñas o datos sensibles,
 - "number" para mostrar un teclado numérico normal y
 - "phone" para obtener el teclado para marcación telefónica
 - "textMultiLine" junto con android:lines(=número de líneas) para cajas de texto multilínea, etc ...

```
<EditText
    android:id="@+id/etEjemplo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="number"
/>
```

```
private lateinit var editTextNumber:
    EditText

....
editTextNumber =
    findViewById(R.id.etEjemplo)
val number=4
editTextNumber.setText(number.toString(
    ))
```



Listeners comunes

- addTextChangedListener- Se dispara cada vez que se cambia el texto en el campo.
- setOnEditorActionListener- Se dispara cuando se presiona un botón de "acción" en el teclado virtual

Se podría adjuntar un oyente a EditText para ejecutar una acción cada vez que se cambie el texto en la vista EditText. Para ello debemos de agregar el objeto TextWatcher como oyente a la referencia de EditText usando addTextChangedListener.

```
editTextNumber.addTextChangedListener(object : TextWatcher {  
  
    override fun afterTextChanged(s: Editable) {  
  
        Log.i("edittext", "after "+s)  
    }  
  
    override fun beforeTextChanged(s: CharSequence, start: Int,  
                                    count: Int, after: Int) {  
  
        Log.i("edittext", String.format("before %s", s))  
    }  
  
    override fun onTextChanged(s: CharSequence, start: Int,  
                                before: Int, count: Int) {  
  
        Log.i("edittext", String.format("ontext %s", s))  
    }  
  
})
```

El objeto TextWatcher está diseñado específicamente para actuar como un detector de cambios de texto

Cuando el objeto TextWatcher se adjunta a un Editable, sus métodos se llamarán cuando se cambie el texto. Por lo tanto, el objeto TextWatcher se puede utilizar como detector de cambios de texto en EditText.

Sus tres métodos principales se invocan de forma secuencial cada vez que el texto cambia:

- beforeTextChanged(): Se llama para indicar que una porción del texto está a punto de ser reemplazada.
- onTextChanged(): Se llama mientras el texto está siendo cambiado.



- `afterTextChanged()`: Se llama después de que el texto ha cambiado, dándote la versión final y editable (Editable) del texto.

En la práctica, la mayoría de las funcionalidades (validación, filtrado, búsqueda) se implementan en `afterTextChanged` porque simplemente necesitan saber cuál es el resultado final del texto.

Button

Se trata de un elemento de la interfaz de usuario en el que el usuario puede tocar o hacer clic para realizar una acción. Para mostrar un botón en una actividad, basta con agregar un botón al archivo XML de diseño de la actividad

```
<Button
    android:id="@+id/button_id"
    android:layout_gravity="center"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/texto_del_boton" />
```

Para especificar una acción cuando se presiona el botón, es necesario crear un `OnClickListener` y añadirlo al objeto del botón en el código de actividad correspondiente:

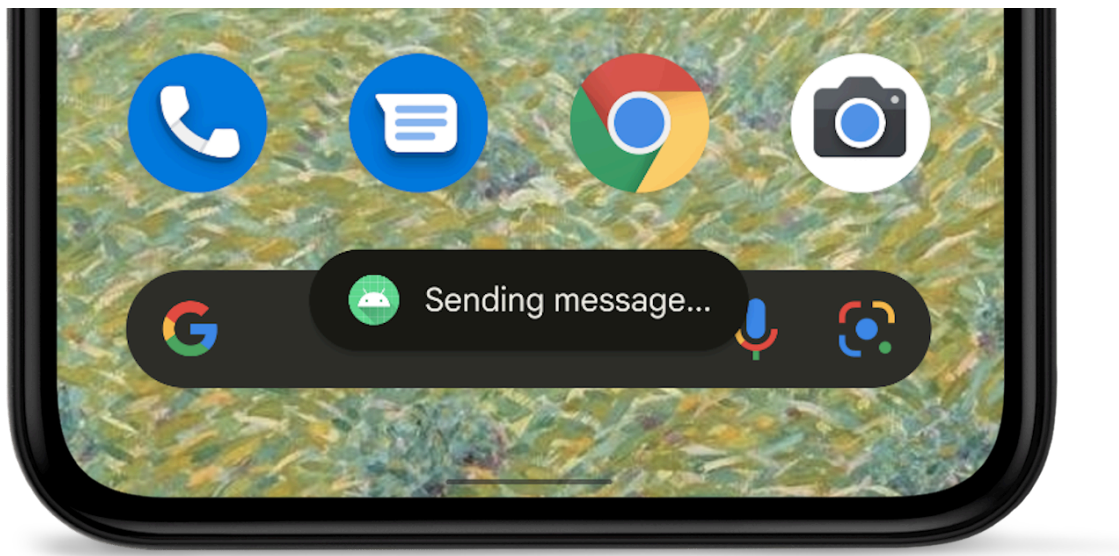
```
boton = findViewById(R.id.button_id)
boton.setOnClickListener(object:View.OnClickListener{
    override fun onClick(v: View?) {
        Log.i("boton", "boton pulsado:")
    }
})
```

Puedes usar la expresión lambda con la función `setOnClickListener()`:

```
boton = findViewById(R.id.button_id)
boton.setOnClickListener{
    Log.i("boton", "boton pulsado:")
}
```

Toast

Toast en Android se utiliza para mostrar un fragmento de texto durante un corto período de tiempo. Cuando se realiza Toast, el fragmento de texto aparece en la pantalla, permanece allí en la pantalla durante aproximadamente 2 o 3 a 5 segundos y desaparece.



```
Toast.makeText(this, "texto a mostrar", Toast.LENGTH_SHORT).show()
```

El método `makeText` necesita contexto, mensaje del sistema y duración del sistema (`LENGTH_SHORT` o `LENGTH_LONG`). Y finalmente, el método `show()` muestra el toast durante la duración especificada.

ImageView

La clase `ImageView` se utiliza para mostrar cualquier tipo de recurso de imagen en la aplicación de Android, ya sea `android.graphics.Bitmap` o `android.graphics.drawable.Drawable` (es una abstracción general para cualquier cosa que se pueda dibujar en Android). La clase `ImageView` o `android.widget.ImageView` hereda la clase `android.view.View`.



Al ser una clase secundaria de View, también podría usarse sus métodos propios, como `setOnClickListener`.

Con el método `setImageResource` podemos cambiar la imagen en vivo.

Logo Vicente Medina:

https://www.murciaeduca.es/iesvicentemedina/sitio/upload/img/Logo_VM-2rt.png

```
logo=findViewById(R.id.imageView)
logo.setOnClickListener { v: View, m: MotionEvent ->
    val action = m.action
    when (action) {
        MotionEvent.ACTION_DOWN -> {
            Log.d("MainActivity", "Down")
        }
        MotionEvent.ACTION_MOVE -> {
            Log.d("MainActivity", "Move")
        }
        MotionEvent.ACTION_UP -> {
            Log.d("MainActivity", "Up")
        }
    }
    true
}
```

RadioGroup y RadioButton

Conjunto de opciones excluyentes entre sí, es decir, tan sólo puedes seleccionar una de ellas a la vez. Los botones de tipo radio (`RadioButton`) se agrupan en un conjunto llamado `RadioGroup`. De esta manera, solo una opción del grupo puede estar activada a la vez. Para crearlo, tan solo tienes que añadir desde tu pantalla de diseño de la actividad, un `RadioGroup` y después arrastrar dentro del `RadioGroup` los `RadioButton` que quieras que se excluyan mutuamente.

En este caso crearemos un `RadioGroup` y un botón. Este último nos informará qué opción ha sido seleccionada cuando se pulse.

```
<RadioGroup
    android:id="@+id/radioGroup1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
```



```
<RadioButton
    android:id="@+id/radioButton1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/RadioButton1"
    android:textSize="24sp" />

<RadioButton
    android:id="@+id/radioButton2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/RadioButton2"
    android:textSize="24sp" />
<RadioButton
    android:id="@+id/radioButton3"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/RadioButton3"
    android:textSize="24sp" />

<RadioButton
    android:id="@+id/radioButton4"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/RadioButton4"
    android:textSize="24sp" />
</RadioGroup>

<Button
    android:id="@+id/button_radio"
    android:layout_gravity="center"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/botóndeRG" />
```

```
radioGroup = findViewById(R.id.radioGroup1)
botonRG=findViewById(R.id.buttonRadio)

botonRG.setOnClickListener {
    //seleccionado = radioGroup.checkedRadioButtonId
    val id_rb: Int =radioGroup.checkedRadioButtonId
```




```
rbSeleccionado =findViewById(id_rb)
Log.d("RadioGroup", rbSeleccionado.text.toString())
}
```

Su oyente podría ser un `setOnCheckedChangeListener`, que recibe el id del `RadioGroup` y de el id de la opción seleccionada.

```
radioGroup.setOnCheckedChangeListener { group, checkedId ->
    Log.d("id radiogroup que lanza",group.toString())
        Log.d("id radioButton que esta
seleccionado",checkedId.toString())
}
```

Switch

Un Switch es un widget de alternancia de dos estados. Los usuarios pueden arrastrar el interruptor hacia adelante y hacia atrás para seleccionar cualquiera de las dos opciones o simplemente tocar el interruptor para alternar entre opciones.

Material Switch

SwitchCompat

Toggle Button

MaterialSwitch es la mejor opción si estás desarrollando una aplicación moderna que sigue las guías de Material Design.

SwitchCompat es útil si necesitas compatibilidad con versiones más antiguas de Android, pero quieres una interfaz más similar a la de un switch moderno.

ToggleButton es una alternativa más antigua y básica, pero generalmente se usa menos en aplicaciones modernas.



A partir de la versión 1.7.0, la nueva clase `MaterialSwitch` reemplazará a la obsoleta clase `SwitchMaterial`. En la mayoría de los casos, debería poder reemplazar todas las referencias a la clase `SwitchMaterial` con `MaterialSwitch` para lograr el aspecto predeterminado.

```
<com.google.android.material.materialswitch.MaterialSwitch
    android:id="@+id/btnSwitch"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:checked="true"
    android:switchPadding="10dp"
    android:text="@string/seleccion" />
```

Su oyente principal es `OnCheckedChangeListener`

```
sw.setOnCheckedChangeListener { buttonView, isChecked ->

    if (isChecked) {
        Toast.makeText(this, "Toggle pulsado a on",
Toast.LENGTH_SHORT).show()
        Log.d("MainActivity", "deslizador ON")
    } else {
        Toast.makeText(this, "Toggle pulsado a OFF",
Toast.LENGTH_SHORT).show()
        Log.d("MainActivity", "deslizador OFF")
    }
}
```

CheckBox

Un control *checkbox* se suele utilizar para marcar o desmarcar opciones en una aplicación, y en Android está representado por la clase del mismo nombre, `CheckBox`

```
<CheckBox android:id="@+id/checkSeleccion"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/marcame"
    android:checked="false" />
```



Los posibles eventos que puede lanzar este control, *onClick* vuelve a ser el más interesante ya que nos indicará cuándo se ha pulsado sobre el checkbox. Dentro de este evento consultaremos normalmente el estado del control con *isChecked* como acabamos de ver.

```
checkboxBox.setOnClickListener {  
    if (it is CheckBox) {  
        if (it.isChecked)  
            checkBox.text = "Checkbox marcado!"  
        else  
            checkBox.text = "Checkbox desmarcado!"  
    }  
}
```

o también:

```
checkboxBox.setOnCheckedChangeListener { buttonView, isChecked ->  
    if (isChecked)  
        buttonView.text = "Checkbox marcado con change!"  
    else  
        buttonView.text = "Checkbox desmarcado con change!"  
}
```